Solution: Find Minimum in Rotated Sorted Array

Problem Statement

You have an array of length `n`, which was initially sorted in ascending order. This array was then rotated `x` times. It is given that 1 <= `x` <= n. For example, if you rotate `[1, 2, 3, 4]` array `3` times, resultant array is `[2, 3, 4, 1]`.

Your task is to find the minimum element from this array. Note that the array contains unique elements.

You must write an algorithm that runs in `O(log n)` time.

Example 1:

- **Input**: [8, 1, 3, 4, 5]
- **Expected Output**: 1
- **Justification**: The smallest number in the array is 1.

Example 2:

- **Input**: [4, 5, 7, 8, 0, 2, 3]
- **Expected Output**: 0
- **Justification**: The smallest number in the array is 0.

Example 3:

- **Input**: [7, 9, 12, 3, 4, 5]
- **Expected Output**: 3
- **Justification**: In this rotated array, the smallest number present is 3.

Solution

To determine the minimum element in a rotated sorted array, we'll leverage the binary search method.

In a standard sorted array, the elements increase (or remain the same) from left to right. But in our rotated sorted array, at some point, there will be a sudden drop, which indicates the start of the original sorted sequence and, hence, the minimum element. This drop will be our guide during the search. With binary search, we'll repeatedly halve our search range until we pinpoint the location of this sudden drop or the smallest element.

1. **Initialization:** Start with two pointers, `left` and `right`, set at the beginning and the end of the array, respectively.

2. **Binary Search Process:**

   - Calculate the middle index, `mid`.
   - If the element at `mid` is greater than the element at `right`, this indicates that the minimum element is somewhere to the right of `mid`. Hence, update `left` to `mid + 1`.
   - Otherwise, the minimum element is to the left, so update `right` to `mid`.

3. **Termination:** The loop will eventually lead `left` to point to the minimum element. This happens when `left` equals `right`.

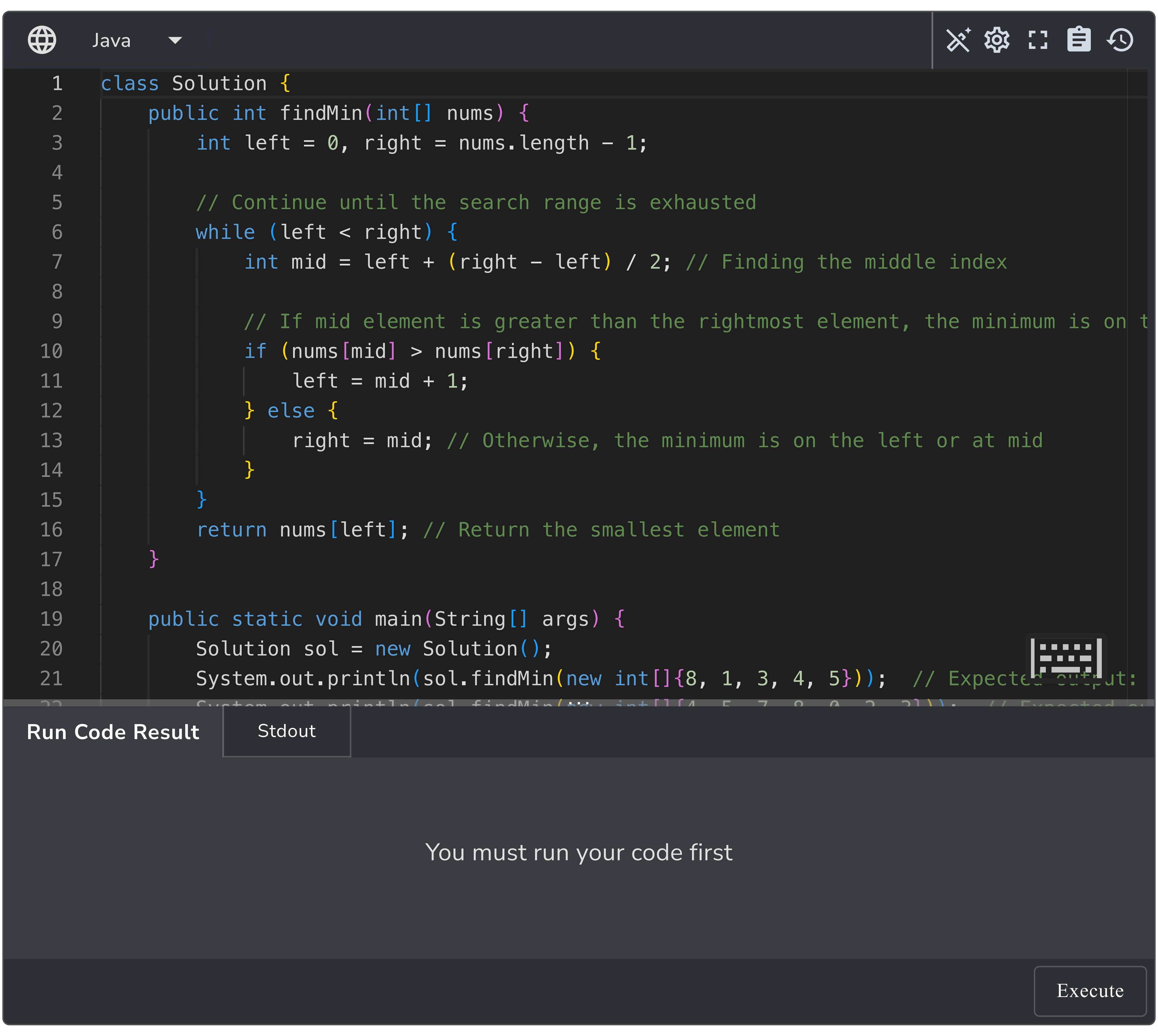
4. **Edge Case Handling:** If the array isn't rotated at all, the smallest element will be the first. Our binary search process accounts for this scenario.

Algorithm Walkthrough

Consider the array [4, 5, 7, 8, 0, 2, 3]:

- Start with `left` = 0 and `right` = 6.
- Calculate `mid` = (0 + 6) / 2 = 3. The array element at index 3 is 8.
- Since 8 > 3 (element at `right`), update `left` to `mid + 1` = 4.
- Now, `left` = 4 and `right` = 6. Calculate `mid` = (4 + 6) / 2 = 5.
- Element at index 5 is 2. Since 2 < 3, update `right` to `mid` = 5.
- Now, `left` = 4 and `right` = 5. Calculate `mid` = (4 + 5) / 2 = 4.
- Element at index 4 is 0. Since 0 < 2, update `right` to `mid` = 4.
- `left` is now equal to `right`, both pointing at index 4, where the minimum element 0 is present.

Code



```java
class Solution {
    public int findMin(int[] nums) {
        int left = 0, right = nums.length - 1;

        // Continue until the search range is exhausted
        while (left < right) {
            int mid = left + (right - left) / 2; // Finding the middle index

            // If mid element is greater than the rightmost element, the minimum is on t
            if (nums[mid] > nums[right]) {
                left = mid + 1;
            } else {
                right = mid; // Otherwise, the minimum is on the left or at mid
            }
        }
        return nums[left]; // Return the smallest element
    }

    public static void main(String[] args) {
        Solution sol = new Solution();
        System.out.println(sol.findMin(new int[]{8, 1, 3, 4, 5}));  // Expected output:
```

You must run your code first

Complexity Analysis

Time Complexity: O(log n) because we are using a binary search approach, which reduces the problem size by half in each step.

Space Complexity: O(1) as we are using a constant amount of space regardless of the input size.

← Previous
Find Minimum In Rotated Sorted Array (Medium)

Next →
Pacific Atlantic Water Flow (Medium)

☐ Mark as Completed